

M4 – Protección de Datos Sensibles y Control de Acceso

Objetivo: Aprender cómo proteger datos sensibles utilizando criptografía, protocolos de seguridad y modelos de control de acceso.

José Picón

INTRODUCCIÓN A LA PROTECCIÓN DE DATOS SENSIBLES

¿Qué son los datos sensibles?

Información confidencial que debe ser protegida contra accesos no autorizados.

Ejemplos:

- ❖ **Datos personales:** Nombre, dirección, teléfono.
- ❖ **Credenciales:** Contraseñas, tokens de sesión.
- ❖ **Datos financieros:** Tarjetas de crédito, cuentas bancarias.

Principales riesgos

- Exposición de datos sin cifrado.
- Fugas de información en tráfico de red.
- Almacenamiento inseguro en bases de datos.

Métodos de protección

- Cifrado de datos (AES, RSA).
- Transmisión segura (TLS).
- Control de acceso basado en roles (RBAC) y atributos (ABAC)

Datos personales sensibles según el RGPD	
Origen étnico o racial	Opiniones políticas
Convicciones religiosas	Convicciones filosóficas
Afiliación sindical	Datos genéticos
Datos biométricos	Datos relativos a la salud
Vida sexual	Orientación sexual

AYUDA Ley Protección de Datos

TRANSMISIÓN SEGURA: TLS

¿Qué es TLS y por qué es importante?



Definición

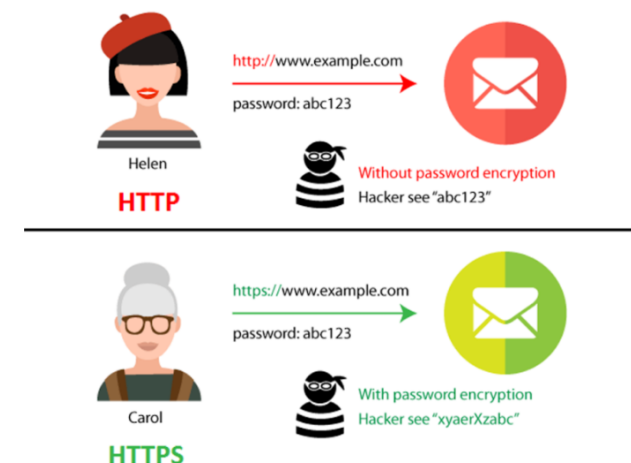
Transport Layer Security (TLS) es un protocolo de seguridad que cifra la comunicación entre clientes y servidores.

Diferencias entre HTTP y HTTPS

- **HTTP:** Comunicación en texto plano, susceptible a ataques.
- **HTTPS:** Utiliza TLS para cifrar la comunicación, protegiendo los datos.

Principales ataques evitados con TLS

- Man-in-the-Middle (*MITM*).
- Escucha de tráfico en redes públicas.



TRANSMISIÓN SEGURA: TLS

Cómo funciona TLS

Proceso de negociación (TLS Handshake)

1. Cliente y servidor intercambian versiones de TLS y algoritmos soportados.
2. Se genera una clave de sesión mediante intercambio de claves (RSA, ECDH).
3. Se establece un canal cifrado con **AES-GCM o ChaCha20-Poly1305**.

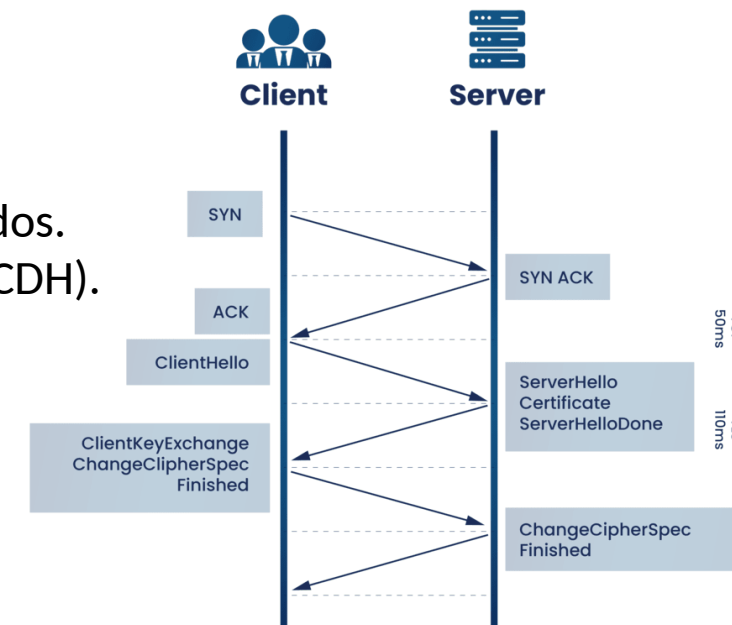
Versiones de TLS

- TLS 1.0 y 1.1 (Obsoletas, inseguras).
- TLS 1.2 (Estándar actual).
- TLS 1.3 (Más seguro y eficiente, elimina cifrados débiles).

Buenas prácticas de configuración TLS

- Deshabilitar TLS 1.0 y 1.1.
- Usar certificados confiables (*Let's Encrypt, DigiCert*).
- Forzar el uso de HTTPS con **HTTP Strict Transport Security (HSTS)**.

https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Strict_Transport_Security_Cheat_Sheet.html



Let's Encrypt

digicert®

ACTIVIDAD

M4-A1-Configuración Segura de TLS

Tema: Protección de datos en tránsito con TLS

Objetivo: Configurar TLS 1.3 en un servidor web para mejorar la seguridad en la comunicación

CIFRADO DE DATOS: AES

Introducción al Cifrado y AES

¿Qué es el cifrado?

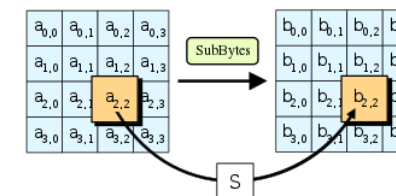
Técnica para proteger datos transformándolos en un formato ilegible sin una clave adecuada.

Tipos de cifrado

- **Simétrico:** Usa la misma clave para cifrar y descifrar (AES).
- **Asimétrico:** Usa un par de claves pública y privada (RSA, ECC).

¿Por qué AES?

- **Advanced Encryption Standard (AES)** es el cifrado simétrico más seguro y utilizado.
- Algoritmos disponibles: **AES-128, AES-192, AES-256**.
- Uso en aplicaciones web, bases de datos y discos cifrados.
- Se utiliza habitualmente en los protocolos VPN



CIFRADO DE DATOS: AES

Funcionamiento de AES

Proceso de cifrado AES

- División del mensaje en bloques de 128 bits.
- Transformaciones matemáticas sobre los bloques usando la clave.
- Modos de operación: ECB (*Electronic CodeBook*), CBC (*Cipher Block Chaining*), GCM (*Galois/Counter Mode*).

Ejemplo de cifrado en Python

```
from Crypto.Cipher import AES
import os
key = os.urandom(32) # Clave AES-256
cipher = AES.new(key, AES.MODE_GCM)
ciphertext, tag = cipher.encrypt_and_digest(b"Mensaje secreto")
```

Buenas prácticas

- Nunca almacenar claves en *código fuente*.
- Usar claves de al menos 256 bits.
- Preferir AES-GCM sobre AES-CBC.

ACTIVIDAD

M4-A2-Cifrado de Datos Sensibles con AES

Tema: Protección de datos en reposo

Objetivo: Usar AES para cifrar y descifrar datos

CONTROL DE ACCESO: RBAC & ABAC

Introducción a los Modelos de Control de Acceso

¿Por qué es necesario un modelo de control de acceso?

- Evita accesos indebidos a recursos sensibles (*Políticas de acceso*)
- Limita los privilegios de cada usuario (*Políticas de privilegios*)

Tipos de control de acceso

- Basado en Roles (*RBAC*).
- Basado en Atributos (*ABAC*).
- Listas de Control de Acceso (*ACLs*).

CONTROL DE ACCESO: RBAC & ABAC

Control de Acceso Basado en Roles (RBAC)

¿Qué es RBAC?

Modelo que asigna permisos según el **Rol** del usuario.

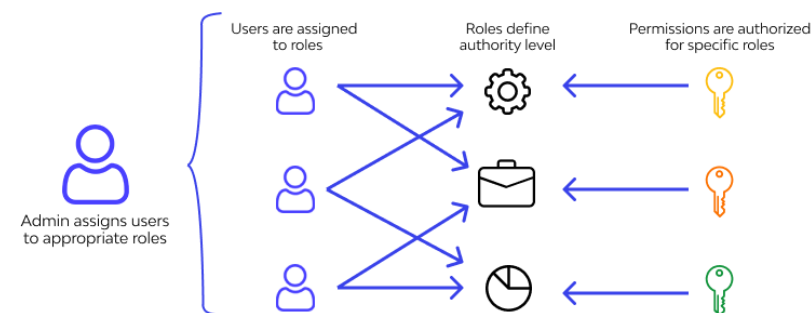
Ejemplo:

- ❖ **Administrador:** Acceso total.
- ❖ **Usuario estándar:** Acceso limitado.
- ❖ **Invitado:** Solo lectura.

Ejemplo en SQL

```
CREATE ROLE admin;  
GRANT SELECT, INSERT, DELETE ON users TO admin;
```

Role-Based Access Control



<https://www.wallarm.com/what/what-exactly-is-role-based-access-control-rbac>

Ventajas y desventajas

- Fácil de implementar en sistemas con roles bien definidos.
- Difícil de escalar si los permisos son muy específicos.

ACTIVIDAD

M4-A3-Implementación de Role-Based Access Control (RBAC)

Tema: Control de acceso basado en roles

Objetivo: Implementar RBAC en una API Flask

CONTROL DE ACCESO: RBAC & ABAC

Control de Acceso Basado en Atributos (ABAC)

¿Qué es ABAC?

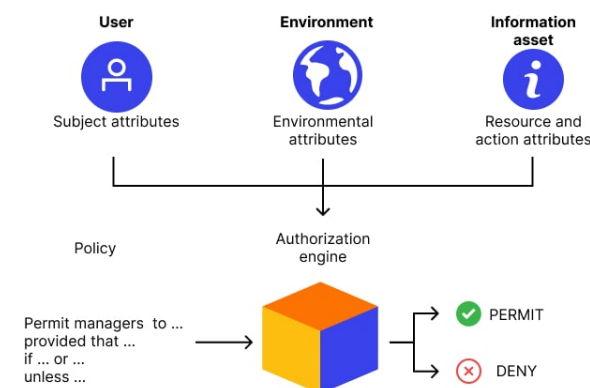
Modelo que usa **atributos** (*edad, ubicación, dispositivo,...*) para tomar decisiones de acceso.

Ejemplo:

- ❖ Solo empleados con **más de 2 años** de antigüedad pueden modificar documentos sensibles.

Ejemplo de política ABAC

```
{  
  "user": "JohnDoe",  
  "resource": "confidential_report.pdf",  
  "conditions": {  
    "role": "manager",  
    "time": "work_hours",  
    "location": "office_network"  
  }  
}
```



Ventajas y desventajas

- Más flexible y dinámico que RBAC.
- Más complejo de implementar y administrar.

ACTIVIDAD

M4-A4-Implementación de Attribute-Based Access Control (ABAC)

Tema: Control de acceso basado en atributos

Objetivo: Implementar ABAC en Flask

CONTROL DE ACCESO: RBAC & ABAC

Comparación RBAC vs. ABAC

Conclusión:

- **RBAC** es ideal para sistemas con *roles estáticos*.
- **ABAC** es mejor para entornos dinámicos con *reglas más complejas*.

Característica	RBAC	ABAC
Basado en	Roles predefinidos	Atributos dinámicos
Escalabilidad	Media	Alta
Ejemplo de uso	Permisos de usuario en un CMS	Acceso condicional a datos en una API

